INFORME DE FIN DE UNIDAD

Datos Generales

Asignatura	Base de datos
Ciclo	3 A
Unidad	1
Resultado de aprendizaje de la unidad	Modela bases de datos de problemas reales, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Nombre del Docente	Edwin René Guamán Quinche
Fecha	08/11/2026
Enlace de Git	https://github.com/carlosamartinezcarrion/Base_Datos_Hotel.git
Nombres y apellidos del estudiante	Carlos Alberto Martinez Carrion

1. TÍTULO

Diseño e implementación de una base de datos relacional para el Sistema de Gestión del Hotel de la ciudad de Loja

2. CONTEXTO DEL PROBLEMA

Un hotel de tamaño mediano ubicado en la ciudad de Loja, con 40 habitaciones de distintos tipos (simple, doble, suite y familiar). Su operación diaria incluye recibir reservas, asignar habitaciones, registrar los servicios que consumen los huéspedes durante su estadía, cobrar y facturar, y llevar el control de la limpieza y las reparaciones de cada habitación.

Hasta ahora las reservas se anotan en un cuaderno y los pagos en hojas de cálculo sueltas. Al no existir una base de datos que centralice la información, se han presentado dobles reservas de una misma habitación, huéspedes frecuentes de los que no se conserva ningún historial y facturas con errores de cálculo. Tampoco es posible saber con certeza qué tarifa se ofreció al reservar cuando los precios cambian, cuántos servicios consumió una reserva ni quién fue el responsable de limpiar o reparar una habitación.

El problema afecta a varios actores: a los huéspedes, que sufren molestias por reservas duplicadas y cobros incorrectos; a los recepcionistas, que deben buscar datos manualmente en cuadernos y hojas dispersas; al personal de limpieza y mantenimiento, cuyo trabajo no queda registrado; y a la gerencia, que carece de información confiable para tomar decisiones sobre ocupación, ingresos y servicios.

Para resolverlo, la base de datos debe gestionar los siguientes datos: huéspedes (cédula o pasaporte, nombres, apellidos, nacionalidad, correo y varios teléfonos); tipos de habitación y sus tarifas; habitaciones con su piso y estado; reservas con una o varias habitaciones y la tarifa por noche vigente al momento de reservar; empleados, diferenciando recepcionistas (turno) y personal de limpieza (piso asignado); catálogo de servicios adicionales y su consumo con el precio vigente en ese momento; facturas y abonos; y el registro de limpieza y mantenimiento. Gestionarlos correctamente es importante porque garantiza la integridad de la información (sin

duplicados ni montos inconsistentes), permite rastrear quién hizo qué y cuándo, y sustenta una facturación transparente y honesta frente al cliente.

3. OBJETIVOS

3.1. Objetivo General

Diseñar e implementar una base de datos relacional para el Hotel de la ciudad de Loja, mediante el modelado entidad-relación, su transformación al modelo relacional y su construcción en SQL sobre MySQL, para centralizar la información de huéspedes, reservas, servicios, pagos y mantenimiento, evitando las dobles reservas y los errores de facturación.

3.2. Objetivos Específicos

- **Objetivo específico 1:** Elaborar el diagrama Entidad-Relación del hotel y derivar de él el modelo relacional de 14 tablas, identificando entidades, atributos, cardinalidades, claves primarias y foráneas, así como la especialización de los empleados en recepcionistas y personal de limpieza.
- **Objetivo específico 2:** Construir el esquema físico en SQL (DDL) con sus restricciones de integridad, poblarlo con datos de ejemplo coherentes (DML) e implementar cinco consultas formuladas en álgebra relacional y traducidas a SQL, que utilicen distintos operadores (selección, proyección, reunión, diferencia, división, unión y agrupamiento).

4. MODELOS CONCEPTUALES DE BASES DE DATOS

4.1 Modelo Entidad-Relación (E-R)

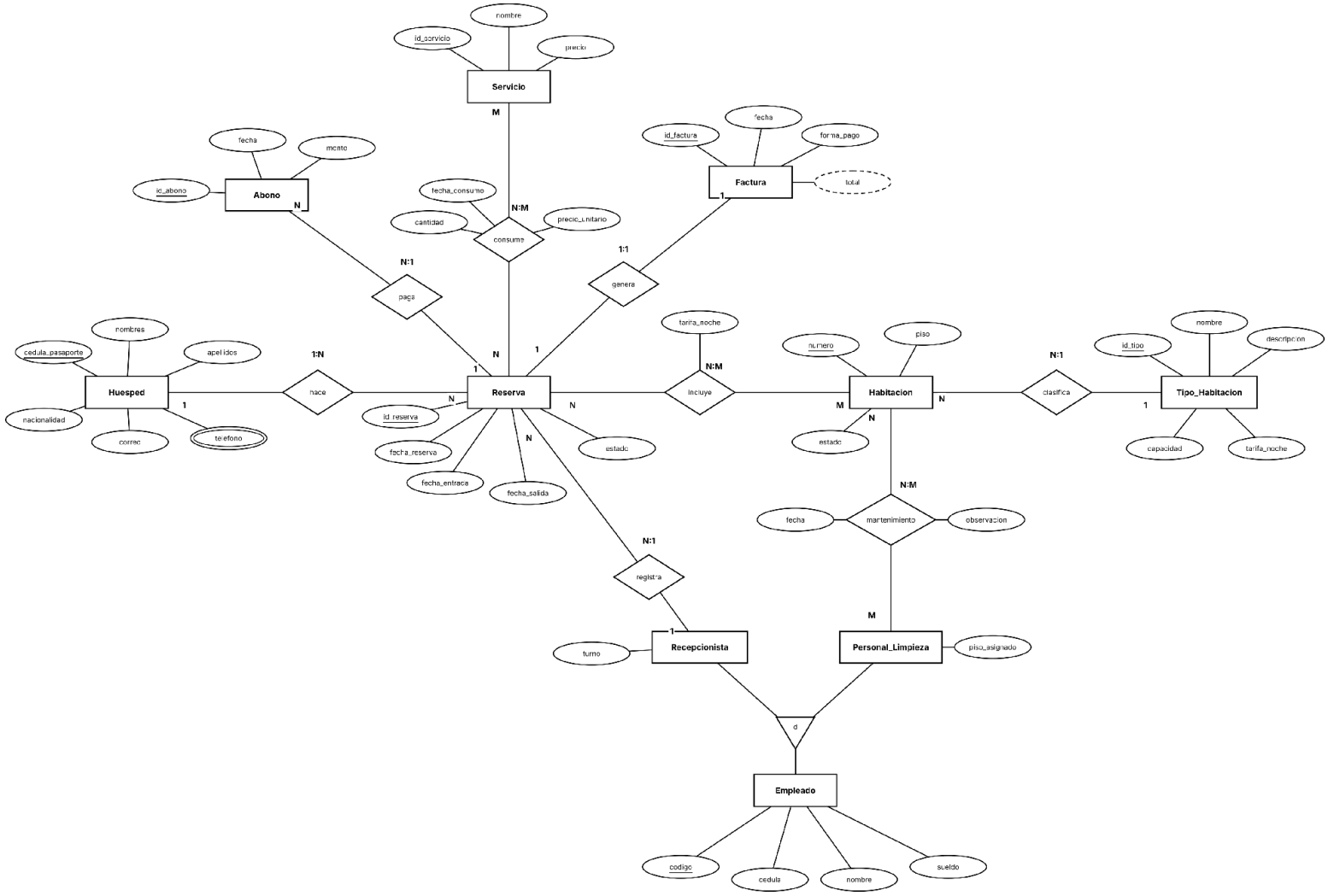


Figura 1. Diagrama Entidad-Relación del sistema de gestión del Hotel.

4.2 Modelo Relacional

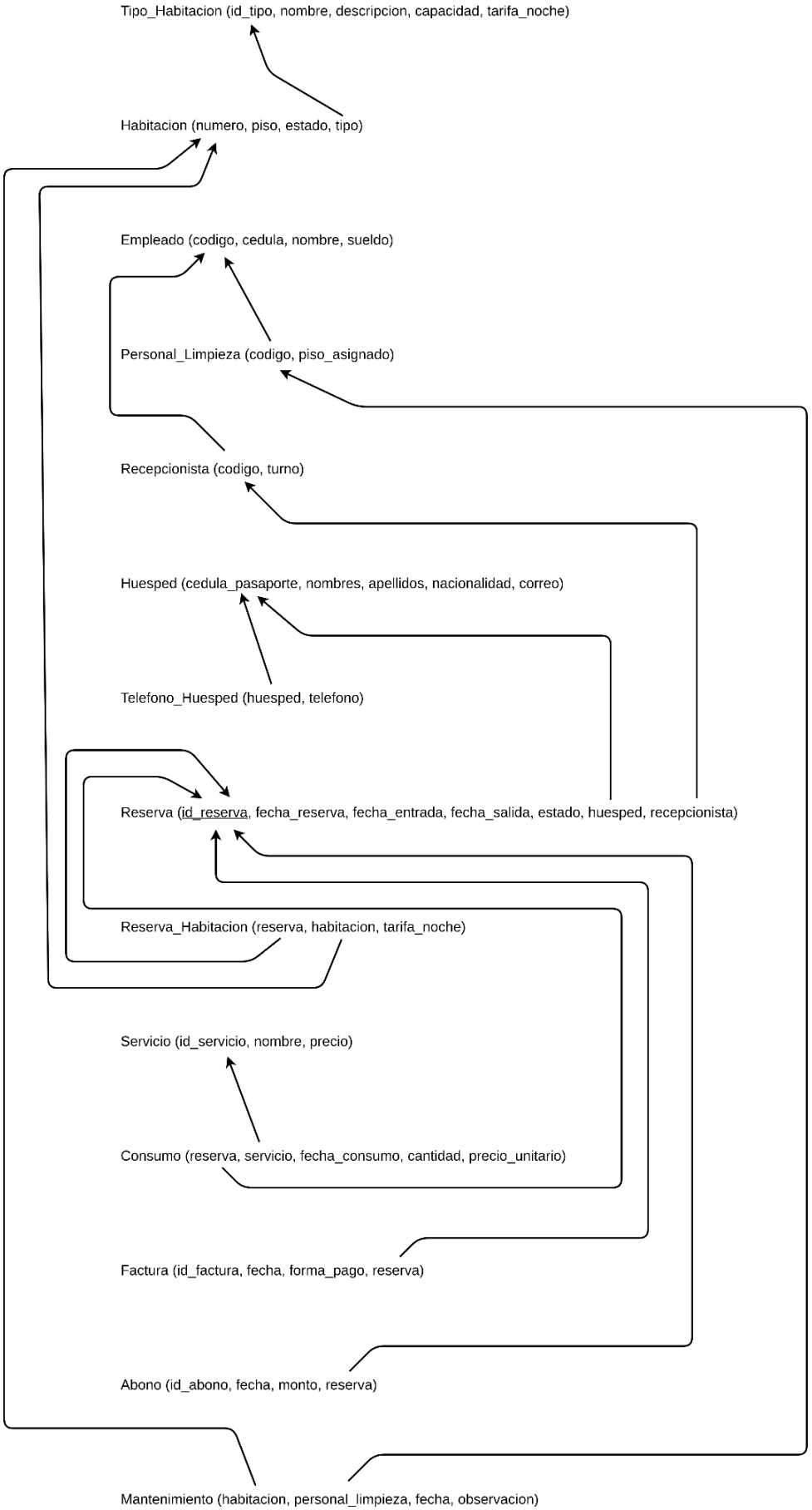


Figura 2. Diagrama relacional del sistema de gestión del Hotel.

CÓDIGO SQL

El código se desarrolló para MySQL 8.0 [5] (compatible con MariaDB 10.x) y se organiza en tres scripts que deben ejecutarse en este orden: 01_schema.sql (DDL), 02_data.sql (DML) y 03_queries.sql (consultas). Convención adoptada: nombres de tablas y columnas en minúsculas con snake_case.

5.1 Creación de Tablas (DDL)

El script crea la base de datos hotel y las 14 tablas del modelo relacional, con claves primarias, claves foráneas, restricciones NOT NULL, UNIQUE, DEFAULT y CHECK. Al final se incluyen objetos adicionales que resuelven problemas del caso: la vista v_total_reserva (calcula el total derivado de cada factura), el disparador trg_evitar_doble_reserva (impide que una habitación se reserve dos veces en fechas que se cruzan) y el disparador trg_factura_solo_finalizada (solo se factura una reserva finalizada).

```
CREATE DATABASE hotel;

USE hotel;

-- 1. TIPO_HABITACION
CREATE TABLE tipo_habitacion (
    id_tipo       INT          NOT NULL AUTO_INCREMENT,
    nombre        VARCHAR(20)   NOT NULL,
    descripcion   VARCHAR(200)  NOT NULL,
    capacidad     TINYINT       NOT NULL,
    tarifa_noche  DECIMAL(8,2)  NOT NULL,
    PRIMARY KEY (id_tipo),
    UNIQUE (nombre)
);

-- 2. HABITACION
CREATE TABLE habitacion (
    numero    SMALLINT     NOT NULL,
    piso      TINYINT      NOT NULL,
    estado    VARCHAR(15)  NOT NULL DEFAULT 'disponible',
    id_tipo   INT          NOT NULL,
    PRIMARY KEY (numero),
    FOREIGN KEY (id_tipo) REFERENCES tipo_habitacion (id_tipo)
);

-- 3. HUESPED
CREATE TABLE huesped (
    cedula_pasaporte  VARCHAR(20)  NOT NULL,
    nombres           VARCHAR(60)  NOT NULL,
    apellidos         VARCHAR(60)  NOT NULL,
    nacionalidad      VARCHAR(40)  NOT NULL,
    correo            VARCHAR(100),
    PRIMARY KEY (cedula_pasaporte),
    UNIQUE (correo)
);

-- 4. TELEFONO_HUESPED (atributo multivaluado de HUESPED)
CREATE TABLE telefono_huesped (
    cedula_pasaporte  VARCHAR(20)  NOT NULL,
    telefono          VARCHAR(20)  NOT NULL,
    PRIMARY KEY (cedula_pasaporte, telefono),
    FOREIGN KEY (cedula_pasaporte)
        REFERENCES huesped (cedula_pasaporte)
);

-- 5. EMPLEADO (superclase) y sus subclases
CREATE TABLE empleado (
    codigo  VARCHAR(10)   NOT NULL,
    cedula  CHAR(10)      NOT NULL,
    nombre  VARCHAR(100)  NOT NULL,
    sueldo  DECIMAL(8,2)  NOT NULL,
    PRIMARY KEY (codigo),
    UNIQUE (cedula)
);

CREATE TABLE recepcionista (
    codigo  VARCHAR(10)  NOT NULL,
    turno   VARCHAR(10)  NOT NULL,
    PRIMARY KEY (codigo),
    FOREIGN KEY (codigo) REFERENCES empleado (codigo)
```

```
);

CREATE TABLE personal_limpieza (
    codigo         VARCHAR(10)  NOT NULL,
    piso_asignado  TINYINT      NOT NULL,
    PRIMARY KEY (codigo),
    FOREIGN KEY (codigo) REFERENCES empleado (codigo)
);

-- 6. RESERVA
CREATE TABLE reserva (
    id_reserva            INT          NOT NULL AUTO_INCREMENT,
    fecha_reserva         DATE         NOT NULL,
    fecha_entrada         DATE         NOT NULL,
    fecha_salida          DATE         NOT NULL,
    estado                VARCHAR(15)  NOT NULL DEFAULT 'pendiente',
    cedula_pasaporte      VARCHAR(20)  NOT NULL,
    codigo_recepcionista  VARCHAR(10)  NOT NULL,
    PRIMARY KEY (id_reserva),
    FOREIGN KEY (cedula_pasaporte)
        REFERENCES huesped (cedula_pasaporte),
    FOREIGN KEY (codigo_recepcionista)
        REFERENCES recepcionista (codigo)
);

-- 7. RESERVA_HABITACION (relación M:N "incluye")
--    tarifa_noche = tarifa congelada al momento de reservar
CREATE TABLE reserva_habitacion (
    id_reserva         INT           NOT NULL,
    numero_habitacion  SMALLINT      NOT NULL,
    tarifa_noche       DECIMAL(8,2)  NOT NULL,
    PRIMARY KEY (id_reserva, numero_habitacion),
    FOREIGN KEY (id_reserva)
        REFERENCES reserva (id_reserva),
    FOREIGN KEY (numero_habitacion)
        REFERENCES habitacion (numero)
);

-- 8. SERVICIO (catálogo de servicios adicionales)
CREATE TABLE servicio (
    id_servicio  INT           NOT NULL AUTO_INCREMENT,
    nombre       VARCHAR(60)   NOT NULL,
    precio       DECIMAL(8,2)  NOT NULL,
    PRIMARY KEY (id_servicio),
    UNIQUE (nombre)
);

-- 9. CONSUMO (relación M:N "consume")
--    precio_unitario = precio vigente al momento del consumo
CREATE TABLE consumo (
    id_reserva       INT           NOT NULL,
    id_servicio      INT           NOT NULL,
    fecha_consumo    DATE          NOT NULL,
    cantidad         INT           NOT NULL DEFAULT 1,
    precio_unitario  DECIMAL(8,2)  NOT NULL,
    PRIMARY KEY (id_reserva, id_servicio, fecha_consumo),
    FOREIGN KEY (id_reserva)
        REFERENCES reserva (id_reserva),
    FOREIGN KEY (id_servicio)
        REFERENCES servicio (id_servicio)
);

-- 10. FACTURA (relación 1:1 con RESERVA; el total es derivado)
CREATE TABLE factura (
    id_factura  INT          NOT NULL AUTO_INCREMENT,
    fecha       DATE         NOT NULL,
    forma_pago  VARCHAR(15)  NOT NULL,
    id_reserva  INT          NOT NULL,
    PRIMARY KEY (id_factura),
    UNIQUE (id_reserva),
    FOREIGN KEY (id_reserva) REFERENCES reserva (id_reserva)
);

-- 11. ABONO (una reserva puede pagarse en varios abonos)
CREATE TABLE abono (
    id_abono    INT           NOT NULL AUTO_INCREMENT,
    fecha       DATE          NOT NULL,
    monto       DECIMAL(10,2)  NOT NULL,
    id_reserva  INT           NOT NULL,
    PRIMARY KEY (id_abono),
    FOREIGN KEY (id_reserva) REFERENCES reserva (id_reserva)
);

-- 12. MANTENIMIENTO (relación M:N entre HABITACION y PERSONAL_LIMPIEZA)
CREATE TABLE mantenimiento (
    numero_habitacion  SMALLINT      NOT NULL,
```

```
    codigo_limpieza    VARCHAR(10)   NOT NULL,
    fecha              DATE          NOT NULL,
    observacion        VARCHAR(200),
    PRIMARY KEY (numero_habitacion, codigo_limpieza, fecha),
    FOREIGN KEY (numero_habitacion) REFERENCES habitacion (numero),
    FOREIGN KEY (codigo_limpieza)   REFERENCES personal_limpieza (codigo)
);
```

5.2 Inserción de Datos (DML)

Se insertaron datos de ejemplo coherentes entre sí en las 14 tablas (mínimo 4 y hasta 22 registros por tabla), de modo que cada clave foránea corresponde a un registro existente. Los datos cubren casos variados: huéspedes ecuatorianos y extranjeros, reservas finalizadas, confirmadas, pendientes y canceladas, reservas con varias habitaciones, tarifas y precios que cambiaron con el tiempo, pagos en varios abonos y huéspedes que nunca consumieron servicios. Los totales de las siete facturas coinciden con la suma de sus abonos.

```
USE hotel;

-- 1. TIPO_HABITACION
INSERT INTO tipo_habitacion (id_tipo, nombre, descripcion, capacidad, tarifa_noche) VALUES
    (1, 'Simple',   'Habitación con una cama de plaza y media, baño privado y TV',           1,  35.00),
    (2, 'Doble',    'Habitación con cama matrimonial o dos camas, baño privado y escritorio',  2,  55.00),
    (3, 'Suite',    'Suite con sala, cama king, jacuzzi y vista a la ciudad',                  2, 110.00),
    (4, 'Familiar', 'Habitación amplia con una cama matrimonial y dos camas individuales',     4,  80.00);

-- 2. HABITACION
INSERT INTO habitacion (numero, piso, estado, id_tipo) VALUES
    (101, 1, 'disponible',    1),
    (102, 1, 'disponible',    1),
    (103, 1, 'disponible',    2),
    (201, 2, 'ocupada',       2),
    (202, 2, 'mantenimiento', 2),
    (203, 2, 'disponible',    4),
    (204, 2, 'disponible',    4),
    (205, 2, 'disponible',    1),
    (301, 3, 'ocupada',       3),
    (302, 3, 'disponible',    3);

-- 3. HUESPED
INSERT INTO huesped (cedula_pasaporte, nombres, apellidos, nacionalidad, correo) VALUES
    ('1104567821', 'Ana Lucía',        'Torres Jaramillo', 'Ecuatoriana',     'ana.torres@correo.ec'),
    ('1103456789', 'Marco Antonio',    'Vélez Cueva',      'Ecuatoriana',     'marco.velez@correo.ec'),
    ('1105678934', 'Sofía Elizabeth',  'Ordóñez Ruiz',     'Ecuatoriana',     'sofia.ordonez@correo.ec'),
    ('1102345678', 'Diego Armando',    'Pineda Lara',      'Ecuatoriana',     'diego.pineda@correo.ec'),
    ('US4587123',  'John Michael',     'Smith',            'Estadounidense',  'john.smith@mail.com'),
    ('CO7712345',  'María Fernanda',   'Gómez Restrepo',   'Colombiana',      'mafe.gomez@mail.com'),
    ('DE55013947', 'Lukas',            'Meyer',            'Alemana',         'lukas.meyer@mail.de'),
    ('IT9087341',  'Valentina',        'Rossi',            'Italiana',        'valentina.rossi@mail.it');

-- 4. TELEFONO_HUESPED (atributo multivaluado)
INSERT INTO telefono_huesped (cedula_pasaporte, telefono) VALUES
    ('1104567821', '0991234567'),
    ('1104567821', '072570123'),
    ('1103456789', '0987654321'),
    ('1105678934', '0998765432'),
    ('1102345678', '0963217854'),
    ('1102345678', '072611456'),
    ('US4587123',  '+1 305 555 0142'),
    ('CO7712345',  '+57 310 555 0187'),
    ('CO7712345',  '+57 601 555 0123'),
    ('DE55013947', '+49 151 5550 1234'),
    ('IT9087341',  '+39 333 555 0166'),
    ('IT9087341',  '+39 06 5550 1877');

-- 5. EMPLEADO (superclase)
INSERT INTO empleado (codigo, cedula, nombre, sueldo) VALUES
    ('E001', '1102345671', 'Patricia Alexandra Cueva Armijos',  520.00),
    ('E002', '1103456782', 'Jorge Luis Armijos Salinas',        500.00),
    ('E003', '1104567893', 'Daniela Estefanía Moreno Castillo', 500.00),
    ('E004', '1105678904', 'Andrés Felipe Jiménez Pardo',       500.00),
    ('E005', '1106789015', 'Karla Michelle Sarango Ochoa',      480.00),
    ('E006', '1107890126', 'Rosa Elena Granda Paladines',       460.00),
    ('E007', '1108901237', 'Luis Fernando Macas Quizhpe',       460.00),
    ('E008', '1109012348', 'Gladys Maribel Zhingre Jaramillo',  460.00),
    ('E009', '1100123459', 'Segundo Manuel Condoy Puglla',      470.00),
    ('E010', '1101234560', 'Martha Isabel Yaguana Ríos',        460.00);

-- 5.1 RECEPCIONISTA
INSERT INTO recepcionista (codigo, turno) VALUES
    ('E001', 'Mañana'),
```

```
    ('E002', 'Tarde'),
    ('E003', 'Noche'),
    ('E004', 'Mañana'),
    ('E005', 'Tarde');

-- 5.2 PERSONAL_LIMPIEZA
INSERT INTO personal_limpieza (codigo, piso_asignado) VALUES
    ('E006', 1),
    ('E007', 1),
    ('E008', 2),
    ('E009', 2),
    ('E010', 3);

-- 6. SERVICIO
INSERT INTO servicio (id_servicio, nombre, precio) VALUES
    (1, 'Desayuno',               6.50),
    (2, 'Lavandería',             8.00),
    (3, 'Spa',                   35.00),
    (4, 'Transporte al aeropuerto', 15.00),
    (5, 'Cena',                  12.00),
    (6, 'Tour por la ciudad de Loja', 25.00);

-- 7. RESERVA (1-7 finalizadas, 8-10 confirmadas, 11 pendiente, 12 cancelada)
INSERT INTO reserva (id_reserva, fecha_reserva, fecha_entrada, fecha_salida, estado, cedula_pasaporte,
codigo_recepcionista) VALUES
    ( 1, '2026-08-28', '2026-09-05', '2026-09-08', 'finalizada', '1104567821', 'E001'),
    ( 2, '2026-09-01', '2026-09-10', '2026-09-12', 'finalizada', '1103456789', 'E002'),
    ( 3, '2026-09-15', '2026-09-20', '2026-09-25', 'finalizada', 'US4587123',  'E001'),
    ( 4, '2026-09-25', '2026-10-01', '2026-10-05', 'finalizada', 'DE55013947', 'E003'),
    ( 5, '2026-10-02', '2026-10-08', '2026-10-10', 'finalizada', '1105678934', 'E002'),
    ( 6, '2026-10-10', '2026-10-15', '2026-10-18', 'finalizada', 'CO7712345',  'E004'),
    ( 7, '2026-10-20', '2026-10-24', '2026-10-27', 'finalizada', 'IT9087341',  'E003'),
    ( 8, '2026-10-25', '2026-11-05', '2026-11-10', 'confirmada', '1102345678', 'E004'),
    ( 9, '2026-10-30', '2026-11-06', '2026-11-12', 'confirmada', 'US4587123',  'E005'),
    (10, '2026-11-01', '2026-11-20', '2026-11-23', 'confirmada', '1104567821', 'E001'),
    (11, '2026-11-05', '2026-12-05', '2026-12-08', 'pendiente',  '1103456789', 'E002'),
    (12, '2026-10-28', '2026-11-15', '2026-11-17', 'cancelada',  '1105678934', 'E004');

-- 8. RESERVA_HABITACION (tarifa congelada al momento de reservar)
INSERT INTO reserva_habitacion (id_reserva, numero_habitacion, tarifa_noche) VALUES
    ( 1, 103,  50.00),
    ( 2, 301, 100.00),
    ( 3, 201,  50.00),
    ( 3, 202,  50.00),
    ( 4, 302, 100.00),
    ( 5, 101,  30.00),
    ( 6, 203,  75.00),
    ( 6, 204,  75.00),
    ( 7, 301, 110.00),
    ( 7, 302, 110.00),
    ( 8, 201,  55.00),
    ( 9, 301, 110.00),
    (10, 203,  80.00),
    (10, 204,  80.00),
    (11, 102,  35.00),
    (12, 205,  35.00);

-- 9. CONSUMO (precio vigente al momento del consumo)
INSERT INTO consumo (id_reserva, id_servicio, fecha_consumo, cantidad, precio_unitario) VALUES
    (1, 1, '2026-09-06', 2,  6.00),
    (1, 1, '2026-09-07', 2,  6.00),
    (1, 2, '2026-09-07', 1,  7.50),
    (2, 5, '2026-09-11', 2, 11.00),
    (2, 4, '2026-09-12', 1, 15.00),
    (3, 1, '2026-09-21', 4,  6.00),
    (3, 2, '2026-09-22', 2,  7.50),
    (3, 3, '2026-09-23', 2, 32.00),
    (3, 4, '2026-09-25', 1, 15.00),
    (4, 1, '2026-10-02', 2,  6.50),
    (4, 3, '2026-10-03', 1, 35.00),
    (4, 4, '2026-10-05', 1, 15.00),
    (6, 1, '2026-10-16', 4,  6.50),
    (6, 6, '2026-10-17', 4, 25.00),
    (7, 1, '2026-10-25', 2,  6.50),
    (7, 5, '2026-10-25', 2, 12.00),
    (7, 3, '2026-10-26', 2, 35.00),
    (7, 2, '2026-10-26', 1,  8.00),
    (8, 1, '2026-11-06', 1,  6.50),
    (8, 1, '2026-11-07', 1,  6.50),
    (9, 5, '2026-11-07', 2, 12.00),
    (9, 6, '2026-11-08', 2, 25.00);

-- 10. FACTURA (solo reservas finalizadas)
INSERT INTO factura (id_factura, fecha, forma_pago, id_reserva) VALUES
    (1, '2026-09-08', 'efectivo',      1),
    (2, '2026-09-12', 'tarjeta',       2),
```

```
    (3, '2026-09-25', 'tarjeta',       3),
    (4, '2026-10-05', 'transferencia', 4),
    (5, '2026-10-10', 'efectivo',      5),
    (6, '2026-10-18', 'tarjeta',       6),
    (7, '2026-10-27', 'transferencia', 7);

-- 11. ABONO (varios abonos por reserva)
INSERT INTO abono (id_abono, fecha, monto, id_reserva) VALUES
    ( 1, '2026-08-28',  60.00, 1),
    ( 2, '2026-09-08', 121.50, 1),
    ( 3, '2026-09-01', 100.00, 2),
    ( 4, '2026-09-12', 137.00, 2),
    ( 5, '2026-09-15', 200.00, 3),
    ( 6, '2026-09-22', 200.00, 3),
    ( 7, '2026-09-25', 218.00, 3),
    ( 8, '2026-09-25', 150.00, 4),
    ( 9, '2026-10-05', 313.00, 4),
    (10, '2026-10-10',  60.00, 5),
    (11, '2026-10-10', 200.00, 6),
    (12, '2026-10-18', 376.00, 6),
    (13, '2026-10-20', 300.00, 7),
    (14, '2026-10-27', 475.00, 7),
    (15, '2026-11-05',  50.00, 8),
    (16, '2026-10-30', 220.00, 9),
    (17, '2026-11-01', 100.00, 10);

-- 12. MANTENIMIENTO (limpieza y reparaciones)
INSERT INTO mantenimiento (numero_habitacion, codigo_limpieza, fecha, observacion) VALUES
    (103, 'E007', '2026-09-08', 'Limpieza posterior al check-out'),
    (201, 'E008', '2026-09-25', 'Limpieza posterior al check-out'),
    (302, 'E010', '2026-10-05', 'Limpieza posterior al check-out'),
    (101, 'E006', '2026-10-10', 'Limpieza profunda posterior a la salida del huésped'),
    (203, 'E008', '2026-10-18', 'Limpieza posterior al check-out'),
    (204, 'E009', '2026-10-18', 'Cambio de bombillos y limpieza'),
    (301, 'E010', '2026-10-27', 'Limpieza y reposición de amenidades'),
    (202, 'E009', '2026-10-28', 'Reparación de fuga en lavamanos'),
    (202, 'E009', '2026-11-02', 'Cambio de grifería; pendiente de revisión final'),
    (102, 'E007', '2026-11-04', 'Mantenimiento preventivo del aire acondicionado');
```

CONSULTA DE ÁLGEBRA RELACIONAL Y TRADUCIR A SQL

Se formularon cinco consultas de distinto nivel de complejidad, primero en álgebra relacional [1], [6] y luego en SQL. Entre todas utilizan los operadores de selección (σ), proyección (π), reunión (⋈), producto cartesiano (×), diferencia (−) y unión (∪). Los resultados descritos se obtuvieron al ejecutar cada consulta sobre los datos de la sección 5.2.

Consulta 1 — Selección (σ), Proyección (π) y Reunión natural (⋈)

Descripción del objetivo de la consulta: Obtener los huéspedes extranjeros (nacionalidad distinta de Ecuatoriana) que tienen reservas no canceladas que incluyen una habitación de tipo Suite, mostrando la habitación, las fechas y la tarifa por noche congelada al reservar. Reúne cinco tablas.

Álgebra Relacional:

$$\pi_{\text{nombres, apellidos, nacionalidad, id_reserva, numero, nombre_tipo, fecha_entrada, fecha_salida, tarifa_noche}} \Big(\sigma_{\text{nacionalidad} \neq \text{'Ecuatoriana'} \text{ and estado} \neq \text{'cancelada'} \text{ and nombre_tipo} = \text{'Suite'}} \Big(\text{HUESPED} \bowtie_{\text{cedula_pasaporte}} \text{RESERVA} \bowtie_{\text{id_reserva}} \text{RESERVA_HABITACION} \bowtie_{\text{numero_habitacion} = \text{numero}} \text{HABITACION} \bowtie_{\text{id_tipo}} \text{TIPO_HABITACION} \Big) \Big)$$

SQL:

```
SELECT h.nombres,
       h.apellidos,
       h.nacionalidad,
       r.id_reserva,
       hb.numero        AS habitacion,
       t.nombre         AS tipo,
       r.fecha_entrada,
       r.fecha_salida,
       rh.tarifa_noche
FROM huesped h
```

```
JOIN reserva           r  ON r.cedula_pasaporte = h.cedula_pasaporte
JOIN reserva_habitacion rh ON rh.id_reserva      = r.id_reserva
JOIN habitacion         hb ON hb.numero          = rh.numero_habitacion
JOIN tipo_habitacion    t  ON t.id_tipo          = hb.id_tipo
WHERE h.nacionalidad != 'Ecuatoriana'
  AND r.estado != 'cancelada'
  AND t.nombre = 'Suite';
```

Resultado esperado: Cuatro filas: Lukas Meyer (suite 302), Valentina Rossi (suites 301 y 302 en la misma reserva) y John Michael Smith (suite 301). Se muestra la tarifa que se pactó al reservar, no la tarifa actual del tipo de habitación.

Consulta 2 — Diferencia (−)

Descripción del objetivo de la consulta: Listar los huéspedes registrados que nunca han consumido un servicio adicional. Se restan, del conjunto de todos los huéspedes, aquellos que aparecen en alguna reserva con consumos.

Álgebra Relacional:

$$\pi_{\text{cedula_pasaporte, nombres, apellidos, correo}} \left(\left(\pi_{\text{cedula_pasaporte}}(\text{HUESPED}) - \pi_{\text{cedula_pasaporte}}(\text{RESERVA} \bowtie_{\text{id_reserva}} \text{CONSUMO}) \right) \bowtie \text{HUESPED} \right)$$

SQL:

```
SELECT h.cedula_pasaporte, h.nombres, h.apellidos, h.correo
FROM huesped h
WHERE NOT EXISTS (SELECT 1
                  FROM reserva r
                  JOIN consumo c ON c.id_reserva = r.id_reserva
                  WHERE r.cedula_pasaporte = h.cedula_pasaporte);
```

Resultado esperado: Una fila: Sofía Elizabeth Ordóñez Ruiz, cuya única reserva finalizada no tuvo consumos y la otra fue cancelada. Es útil para campañas que promuevan los servicios del hotel.

Consulta 3 — Producto cartesiano (×), Diferencia (−) y Proyección (π)

Descripción del objetivo de la consulta: Encontrar los huéspedes que han consumido todos los servicios del catálogo (desayuno, lavandería, spa, transporte, cena y tour). Es una división relacional, que se expresa con los operadores básicos: se forman todos los pares huésped-servicio posibles con el producto cartesiano, se restan los pares que sí ocurrieron, y los huéspedes que quedan con algún par faltante se restan de todos los huéspedes. En SQL se traduce con un doble NOT EXISTS: no existe un servicio del catálogo que el huésped no haya consumido.

Álgebra Relacional:

$$\text{CONSUMIDOS} = \pi_{\text{cedula_pasaporte, id_servicio}} \left(\text{RESERVA} \bowtie_{\text{id_reserva}} \text{CONSUMO} \right)$$

$$\text{FALTANTES} = \left(\pi_{\text{cedula_pasaporte}}(\text{HUESPED}) \times \pi_{\text{id_servicio}}(\text{SERVICIO}) \right) - \text{CONSUMIDOS}$$

$$\pi_{\text{nombres, apellidos}} \left(\text{HUESPED} \bowtie \left(\pi_{\text{cedula_pasaporte}}(\text{HUESPED}) - \pi_{\text{cedula_pasaporte}}(\text{FALTANTES}) \right) \right)$$

SQL:

```
SELECT h.cedula_pasaporte, h.nombres, h.apellidos
FROM huesped h
WHERE NOT EXISTS (SELECT 1
                  FROM servicio s
                  WHERE NOT EXISTS (SELECT 1
                                    FROM reserva r
                                    JOIN consumo c ON c.id_reserva = r.id_reserva
                                    WHERE r.cedula_pasaporte = h.cedula_pasaporte
```

```
                                    AND c.id_servicio      = s.id_servicio));
```

Resultado esperado: Una fila: John Michael Smith, quien consumió desayuno, lavandería, spa y transporte en su primera reserva, y cena y tour en la segunda.

Consulta 4 — Unión (∪)

Descripción del objetivo de la consulta: Conformar la lista del personal de refuerzo: recepcionistas del turno de noche junto con el personal de limpieza asignado a los pisos 2 y 3. Ambos subconjuntos se proyectan con los mismos atributos para que sean compatibles con la unión; en SQL se agrega una columna literal rol para identificar el origen de cada fila.

Álgebra Relacional:

$$\pi_{\text{codigo, nombre}} (\sigma_{\text{turno} = \text{'Noche'}} (\text{EMPLEADO} \bowtie_{\text{codigo}} \text{RECEPCIONISTA})) \cup \pi_{\text{codigo, nombre}} (\sigma_{\text{piso_asignado} \geq 2} (\text{EMPLEADO} \bowtie_{\text{codigo}} \text{PERSONAL_LIMPIEZA}))$$

SQL:

```
SELECT e.codigo, e.nombre, 'Recepcionista' AS rol, CONCAT('Turno ', rc.turno) AS detalle
FROM empleado e
JOIN recepcionista rc ON rc.codigo = e.codigo
WHERE rc.turno = 'Noche'
UNION
SELECT e.codigo, e.nombre, 'Personal de limpieza', CONCAT('Piso ', pl.piso_asignado)
FROM empleado e
JOIN personal_limpieza pl ON pl.codigo = e.codigo
WHERE pl.piso_asignado >= 2;
```

Resultado esperado: Cuatro empleados: tres del personal de limpieza (E008, E009 y E010) y la recepcionista del turno de noche (E003). UNION elimina duplicados, como la unión del álgebra relacional.

Consulta 5 — Selección (σ), Proyección (π) y Reunión natural (⋈)

Descripción del objetivo de la consulta: Identificar los consumos de servicios cuyo precio pactado al momento del consumo es distinto del precio actual del catálogo, mostrando el huésped, la reserva y ambos precios. Permite ver qué facturas se conservaron con el precio antiguo después de un cambio de tarifas. Reúne cuatro tablas.

Álgebra Relacional:

$$\pi_{\text{nombres, apellidos, id_reserva, nombre_servicio, fecha_consumo, precio_unitario, precio}} (\sigma_{\text{precio_unitario} \neq \text{precio}} (\text{HUESPED} \bowtie_{\text{cedula_pasaporte}} \text{RESERVA} \bowtie_{\text{id_reserva}} \text{CONSUMO} \bowtie_{\text{id_servicio}} \text{SERVICIO}))$$

En el esquema físico, el atributo nombre de SERVICIO se muestra aquí como nombre_servicio.

SQL:

```
SELECT h.nombres,
       h.apellidos,
       r.id_reserva,
       s.nombre          AS servicio,
       c.fecha_consumo,
       c.precio_unitario AS precio_pactado,
       s.precio          AS precio_actual
FROM servicio s
JOIN consumo c ON c.id_servicio = s.id_servicio
JOIN reserva r ON r.id_reserva  = c.id_reserva
JOIN huesped h ON h.cedula_pasaporte = r.cedula_pasaporte
WHERE c.precio_unitario != s.precio;
```

Resultado esperado: Siete filas, de tres huéspedes: Ana Lucía Torres (desayuno y lavandería, reserva 1), Marco Antonio Vélez (cena, reserva 2) y John Michael Smith (desayuno, lavandería y spa, reserva 3). En todos los casos el precio pactado fue menor que el actual (por ejemplo, el spa se consumió a $32.00 y hoy cuesta $35.00).

CONCLUSIONES

Conclusión 1 — Modelado conceptual. La elaboración del diagrama E-R y su transformación al modelo relacional permitió entender que el diseño correcto de la base de datos se decide antes de escribir cualquier sentencia SQL. Las principales dificultades estuvieron en distinguir cuándo una relación debía convertirse en una tabla propia: la relación “incluye” entre reserva y habitación, la relación “consume” entre reserva y servicio y la relación de mantenimiento son N:M y además tienen atributos (tarifa_noche, precio_unitario, fecha, observacion), lo que exigió tablas intermedias con clave primaria compuesta. También requirió criterio modelar el atributo multivaluado telefono, el atributo derivado total de la factura (que no se almacena) y la especialización disjunta de Empleado en recepcionistas y personal de limpieza.

Conclusión 2 — Implementación SQL. Escribir el DDL y el DML hizo visible cómo las restricciones de integridad (PRIMARY KEY, FOREIGN KEY, UNIQUE y DEFAULT) trasladan las reglas del negocio a la propia base de datos: estados y formas de pago válidos, fechas coherentes, una factura por reserva y montos positivos. Resultaron especialmente relevantes las claves foráneas, porque obligan a insertar los datos en el orden correcto y garantizan la integridad referencial, y los disparadores, que resuelven el problema central del hotel al impedir que una misma habitación se reserve dos veces en fechas que se cruzan. Guardar la tarifa y el precio vigentes en reserva_habitacion y consumo, en lugar de consultarlos del catálogo, evita que los cambios de precios alteren facturas ya emitidas.

Conclusión 3 — Álgebra relacional. Formular cada consulta primero en álgebra relacional ayudó a razonar qué tablas reunir, qué condiciones aplicar y qué columnas proyectar antes de escribir las sentencias SQL. También permitió comprender que operaciones como la diferencia y la división no tienen una instrucción directa en SQL, sino que pueden expresarse mediante NOT EXISTS y doble NOT EXISTS, respectivamente. Las consultas desarrolladas resultaron útiles para el sistema del hotel, ya que permiten identificar huéspedes extranjeros con reservas de suites, huéspedes que nunca han consumido servicios, empleados disponibles para reforzar determinadas áreas y diferencias entre los precios pactados y los actuales. De esta manera, se comprobó cómo el álgebra relacional facilita la obtención de información específica y contribuye a una mejor gestión de los datos.

BIBLIOGRAFÍA

[1]A. Silberschatz, H. F. Korth y S. Sudarshan, Database System Concepts, 7.ª ed. Nueva York, NY, EE. UU.: McGraw-Hill Education, 2019.

[2]R. Elmasri y S. B. Navathe, Fundamentals of Database Systems, 7.ª ed. Hoboken, NJ, EE. UU.: Pearson, 2016.

[3]E. F. Codd, “A relational model of data for large shared data banks,” Communications of the ACM, vol. 13, n.º 6, pp. 377–387, jun. 1970.

[4]P. P.-S. Chen, “The entity-relationship model—toward a unified view of data,” ACM Transactions on Database Systems, vol. 1, n.º 1, pp. 9–36, mar. 1976.

[5]Oracle Corporation, “MySQL 8.0 Reference Manual.” dev.mysql.com. [En línea]. Disponible: https://dev.mysql.com/doc/refman/8.0/en/ (consultado el 8 de nov. de 2026).

[6]C. J. Date, An Introduction to Database Systems, 8.ª ed. Boston, MA, EE. UU.: Addison-Wesley, 2003.

[7]JGraph Ltd., “draw.io (diagrams.net).” [En línea]. Disponible: https://www.drawio.com (consultado el 8 de nov. de 2026).